协议

用途：直接按本文实现串口解析与每步执行逻辑

1. 内容

1. 视觉会发什么。
2. 电机每步前怎么取和怎么用。
3. 出问题时怎么安全降级。

2. 当前必须接入的消息

1. `LINE_CTRL` （`MSG_TYPE = 0x02`），8~12Hz（建议10Hz）。
2. `HEARTBEAT` （`MSG_TYPE = 0x01`），10Hz。
3. `QR_EVENT`、`OBSTACLE_EVENT` 属于事件流，触发才发，不是每帧都发。

3. 串口与通用帧格式

串口参数：

- UART1
- 115200
- 8N1
- 小端（Little Endian）

所有消息统一帧结构：

字段	长度(字节)	类型	说明
SOF1	1	u8	固定 `0x55`
SOF2	1	u8	固定 `0xAA`
VER	1	u8	当前固定 `0x02`
MSG_TYPE	1	u8	消息类型
FLAGS	1	u8	bit0= `ACK_REQ`，bit1= `IS_ACK`

字段	长度(字节)	类型	说明
SEQ	1	u8	0~255循环
TS_MS	4	u32	发送时间戳（ms）
LEN	1	u8	Payload长度
PAYLOAD	LEN	bytes	负载
CRC16	2	u16	校验范围： `VER..PAYLOAD`

CRC16参数（CCITT-FALSE）：

- poly= `0x1021`
- init= `0xFFFF`
- xorout= `0x0000`

4. LINE_CTRL（主控制消息）

消息类型： `0x02`
Payload长度： `12` 字节

偏移	字段	类型	含义	取值/单位
0	mode_u8	u8	视觉模式	`0=idle, 1=line_follow, 2=lost_search`
1	conf_u8	u8	置信度	`0~100`
2	lost_u8	u8	是否丢线	`0/1`
3	route_u8	u8	方向类别	`1=go, 2=left, 3=right, 4=slight_left`
4~5	ex_mm_i16	i16	横向偏差	mm
6~7	ang_cdeg_i16	i16	航向误差	cdeg（0.01度）
8~9	v_cmd_mmps_i16	i16	线速度指令	mm/s，P1可为0
10~11	w_cmd_mradps_i16	i16	角速度指令	mrad/s，P1可为0

4.1 电机侧本阶段主用字段

主用： `mode + route + conf + lost`
辅助调试： `ex + ang`
可先忽略： `v_cmd + w_cmd` （本阶段可能发送0，因为不知道能不能做到。）

4.2 route不是“力度”，只是“方向”

- `route=1` ： 直行
- `route=2` ： 左
- `route=3` ： 右
- `route=4` ： 微左

转向力度由ex和ang决定，不靠route细分很多档。

5. HEARTBEAT（在线保活）

消息类型： `0x01`
建议Payload长度： `1` 字节（仅 `mode_u8` ）

说明：

- `SEQ` 、 `TS_MS` 已在帧头，不必在payload重复。
- HEARTBEAT频率固定10Hz。

6. 事件消息与ACK

用于二维码、障碍、模式切换等“必须可靠到达”的信息。

MSG_TYPE	名称	可靠性
`0x10`	QR_EVENT	ACK + 超时重发
`0x11`	OBSTACLE_EVENT	ACK + 超时重发
`0x12`	MODE_SWITCH_REQ	ACK + 超时重发
`0x80`	ACK/NACK	应答
`0xE0`	ESTOP	最高优先级

ACK策略：

- 超时 `80ms` 重发
- 最多 `3` 次

7. 每步执行规则（电机侧直接照做）

1. 每次迈步前，读取“最新一帧有效LINE_CTRL”。
2. 如果一个步周期内收到了多帧，可选：最近3帧route做多数投票，conf取中值。
3. 如果 `lost=1` 或 `conf<35`： 本步不前推，只做原地小角度调整或停步。
4. 如果 `35<=conf<60`： 仅允许小步修正。
5. 如果 `conf>=60`： 按route正常执行当前步。

安全超时：

- `300ms` 内无有效 `LINE_CTRL/HEARTBEAT -> SAFE_STOP`
- `800ms` 仍未恢复 `-> LOST_RECOVERY`

8. 参考伪代码（AI写的）

```
on_uart_bytes(data):
  frame = parse_frame(data)
  if frame.invalid_crc or frame.ver != 0x02:
    return

  update_last_rx_time_ms(frame.ts_ms)
  update_latest_seq(frame.seq)

  if frame.msg_type == 0x02:  # LINE_CTRL
    msg = parse_line_ctrl(frame.payload)
    cache_latest_line_ctrl(msg, frame.seq, frame.ts_ms)

  elif frame.msg_type == 0x01:  # HEARTBEAT
    cache_heartbeat(frame.seq, frame.ts_ms)

  elif frame.msg_type in [0x10, 0x11, 0x12, 0xE0]:
    handle_event(frame)
    send_ack(frame.seq, frame.msg_type, code=0)


before_each_step():
  if now_ms - last_valid_rx_ms > 300:
    enter_safe_stop()
    return

  msg = get_latest_valid_line_ctrl()  # 可选: 最近3帧route多数投票
  if msg is None:
    enter_safe_stop()
    return

  if msg.lost == 1 or msg.conf < 35:
    do_in_place_adjust_or_stop()
    return

  if msg.conf < 60:
    do_small_step_by_route(msg.route)
  else:
    do_normal_step_by_route(msg.route)
```

9. 联调时最小检查项

1. 能稳定解析 `LINE_CTRL` 与 `HEARTBEAT` 。
2. `route` 映射不出错（1/2/3/4）。
3. `conf/lost` 阈值逻辑按第7节执行。
4. `300ms` 超时停机有效。
5. 事件消息ACK可回、重发可收。

10. 一条LINE_CTRL示例

设定：

- mode=1
- conf=70
- lost=0
- route=2
- ex=-10mm
- ang=300cdeg
- v_cmd=0
- w_cmd=0

12字节payload（hex）：

```
01 46 00 02 F6 FF 2C 01 00 00 00 00
```